

ACSD: An Executable Profile for Anonymous Scholarly Releases, Series Lineage, and Cyclic Citations

Anonymous authors

Abstract

Anonymous scholarly disclosure needs a more precise answer than “put a signed PDF in a repository.” A signature identifies a signing key, not necessarily a person; a repository timestamp is not independent time; and a collection of mutually citing papers appears circular if every reference is a content hash. This paper presents ACSD, a narrow executable profile for anonymous scholarly releases. A release binds exact content bytes, ordered anonymous author slots, contribution and AI-use declarations, individual COSE endorsements, and signed byte-level citation witnesses. A separate optional series layer binds stable random `WorkIDs` to completed content-addressed releases. This separates a semantic citation graph, which may contain cycles, from a cryptographic commitment graph, which is acyclic. The profile also distinguishes legitimate branches, verifiable equivocation, and content reuse under a different lineage, rather than pretending that a valid signature proves originality. Our prototype is an executable command-line tool with Ed25519 COSE endorsements and RFC 3161 timestamping, a 64-vector Python–Node canonical-JSON differential harness, negative fixtures, and an axiom-free Lean model of its acceptance contracts. We report what these mechanisms establish and, equally importantly, what they do not establish: natural-person identity, contribution truth, originality, peer review, or legal nonrepudiation.

1 Introduction

Researchers often want to disclose a manuscript before conventional publication while keeping the author identity unavailable during review. A common informal proposal is to upload an anonymous file, sign it, and rely on repository history. That proposal leaves several distinct questions entangled.

One concrete trigger is the interval around an arXiv submission: a first-time or new-category submission may require endorsement, while moderation and review are separate from scientific peer review. During that interval an author may want a public, content-anonymous evidence package that preserves exact bytes and team assent without pretending to replace arXiv’s process. We use this as an application scenario, not as a claim that arXiv queues have a measured backlog or that ACSD should circumvent its rules.

1. What exact bytes did one or more keys approve?
2. Can a later verifier distinguish a revision, a legitimate branch, and an equivocation?
3. Can a sequence of papers cite one another, including mutual citations, without a circular content-hash dependency?
4. What does a time witness establish, and what does it not establish?

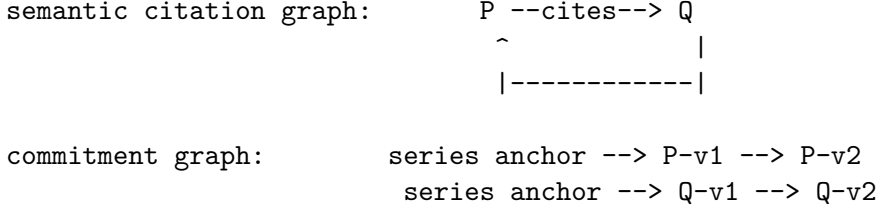


Figure 1: ACS D separates cyclic semantic citations from acyclic exact-byte commitments. Stable random WorkIDs support the first graph; release digests and predecessor links support the second.

The first question is about key control and integrity. It is not, by itself, a proof that a particular natural person wrote the work. The second and third are data-model and verification questions. The fourth requires an external witness such as an RFC 3161 timestamp authority (TSA), not a local clock or a mutable Git field. Conflating these layers produces systems that overclaim.

This paper presents ACS D (Anonymous Scholarly Claim and Disclosure), a deliberately narrow application profile. It uses standard content hashing, COSE signatures, and provenance concepts rather than proposing a new primitive. The profile is designed for content-anonymous release: the public paper and artifact do not name authors, but an account hosting them can still be linkable. It is therefore *not* described as author-unlinkable double blind publication.

The main contributions are:

1. a self-contained paper-release object with ordered anonymous author slots and per-author endorsements;
2. a two-layer construction that permits cyclic scholarly citations over stable WorkIDs while keeping content commitments acyclic;
3. signed UTF-8 byte-range citation witnesses and a restricted canonical JSON profile to reduce field-interpretation ambiguity;
4. explicit classifications for branches, equivocation, and copied content under a fresh lineage; and
5. an executable prototype with cross-implementation verification, negative fixtures, and a small axiom-free Lean model of selected acceptance conditions.

2 Scope and Threat Model

Assets. A release contains manuscript bytes, an ordered list of anonymous author keys, roles and contribution declarations, an AI-use declaration, and optional parent-release information. A series package contains references to completed releases, a series key, sequencing information, and cross-paper semantic relations. Public keys are pseudonymous identifiers.

Adversary. We consider a party that can copy public content, generate fresh keys, reorder or substitute unsigned metadata, present a signed but malformed JSON payload, and claim unsupported cross-paper citations. We also consider a series key that signs two incompatible objects. We do not assume that a signature reveals a human identity, that a contribution declaration is factually true, or that a local timestamp is trusted.

Non-goals. ACSD does not prove authorship by a natural person, independent discovery, plagiarism, venue acceptance, peer review, or legal nonrepudiation. These are not merely unimplemented features: some require social, institutional, or legal evidence outside the cryptographic object. The profile instead preserves evidence that can be useful in a later dispute.

3 Release Layer

3.1 Canonical release object

Each *PaperRelease* has a random stable *WorkID*, a visible slot (*WorkID*, *version*, *line*), a content path, content length and digest, ordered author records, declared roles, contributions, an AI-use declaration, optional exact predecessor, and a list of citation witnesses. The release identifier is

$$\text{ReleaseID} = \text{urn:sha256:H}(\text{canonical}(R)).$$

The content digest is over exact manuscript bytes. The release object uses a deliberately restricted JSON encoding: recursively sorted ASCII object keys, no insignificant whitespace, and safe integers only. A verifier parses, re-canonicalizes, and compares bytes before accepting a signature. This is a profile rule, not a claim that it is a general JSON standard.

Each listed author key separately issues a COSE Sign1 endorsement over the same canonical release bytes. Acceptance requires the expected endorsements, matching public keys, matching content digest, and all field constraints. A verifier can then state: “these pseudonymous keys approved these exact release bytes.” It cannot infer who controls a key without separate enrollment evidence.

3.2 Citation witnesses

The prototype recognizes only literal occurrences of a canonical token `urn:uuid:<work-id>` in the UTF-8 manuscript bytes. For every occurrence it records and signs

$$(\text{WorkID}, \text{byteOffset}, \text{byteLength}).$$

The published `reference_work_ids` list must equal the ordered de-duplicated projection of these witnesses. This prevents a series package from inventing a citation edge that has no corresponding token in its source release, and prevents a copied text from silently relabeling an old target as a new *WorkID*.

The rule intentionally does not parse PDFs, infer references from natural language, or determine whether a citation is intellectually appropriate. It is an exact byte-binding contract, not a semantic understanding claim.

For example, one UTF-8 source line contains the following token:

`urn:uuid:12345678-1234-1234-1234-123456789abc`

The witness records that *WorkID*, the byte offset of the first character of the token, and its byte length (45 bytes for this token). An editor that inserts a character before the token must regenerate the witness. This is intentionally mechanical: a source-level helper or build macro can insert the token, but v1 does not yet provide such editor integration, and hiding the token in a PDF would defeat this scanner.

A minimal release fragment is conceptually:

```
{
  "slot": {
    "line": "main",
    "version": 1,
    "work_id": "urn:uuid:...",
    "content": {
      "sha256": "...",
      "reference_work_ids": ["urn:uuid:..."],
      "citation_witnesses": [
        {
          "work_id": "urn:uuid:...",
          "byte_offset": 35,
          "byte_length": 45
        }
      ]
    }
  }
}
```

The ellipses above are presentation placeholders only. The released demo fixture binds a real manuscript’s bytes, digest, and work identifier (the governance and PEC objects are real, not placeholders), using synthetic author identifiers. Real series releases are produced by the v1 fixture generator.

4 Series Layer and Cyclic Citations

4.1 Two graphs, two purposes

A paper remains verifiable after all series packages are removed. The optional series layer records relationships among independently complete releases. A signed series package contains

$$B_k = \text{Sign}_S(\text{seriesId}, \text{epoch}, \text{sequence}, \text{previousBundleHash}, \\ \text{rotationHash}, \text{rank}, \text{releases}, \text{heads}, \text{semanticEdges}).$$

The semantic graph has vertices `WorkIDs` and may contain $P \rightarrow Q$ and $Q \rightarrow P$. Its edges express declared citation or related-work relations. The commitment graph has series anchors and completed `ReleaseIDs`. It does not make a release depend on a future series package. Thus two authors can cite stable random `WorkIDs` before both final documents are content-addressed; a later package resolves each `WorkID` to a completed `ReleaseID`. The semantic graph may be cyclic while the commitment graph remains a DAG.

The package includes a signed structural rank strictly above every release version it resolves. Combining anchor-to-release edges with exact release-to-parent edges decreases this rank along a commitment path. This rank is a graph certificate, not a timestamp, publication date, or priority claim.

The package fields have narrow roles: *epoch* names the active series key generation; *rotationHash* commits its authorized key transition; *heads* maps each `WorkID` to its advertised line head; and *semanticEdges* records declared citation or related-work relations. The rank is only a finite measure certifying that the commitment graph has no cycle.

4.2 Revisions, branches, and conflicts

A revision names one exact predecessor. Multiple children of a parent are allowed, because a main manuscript and a computational appendix can be legitimate independent lines. A paper-level equivocation is instead detected when two valid, different releases occupy the same exclusive slot. A series-level equivocation is detected when two valid packages share the same series, epoch, sequence, and predecessor hash while committing to different contents.

An adversary can copy the content bytes, issue a fresh `WorkID` and keys, and obtain a valid new lineage. ACSD classifies this as content reuse under a different lineage, not as a cryptographic proof of plagiarism. That restraint is important: reuse may be authorized, collaborative, or independently derived.

As a concrete attack, suppose an attacker copies P-v1, changes its `WorkID`, and asks the series package to claim that it cites Q. The release verifier recomputes the content digest and byte

witnesses; if the copied source does not contain the declared Q token, the package is rejected as an unsupported edge. If the attacker copies the token too, the package can still be valid under a new lineage. The evidence then says only “content reuse under a different lineage”; deciding plagiarism or authorization remains outside the verifier.

5 Time Evidence

Time is intentionally external to release signing. An RFC 3161 request can contain the SHA-256 imprint of a frozen release or a frozen manifest. A valid response, verified against a pinned TSA trust anchor and a nonce-bearing request, supports the limited statement that the TSA saw that imprint no later than its recorded `genTime`. The prototype includes a separate adapter that constructs and verifies this receipt format, including raw request and response preservation. Its local test TSA is only a protocol test and is not presented as independent time evidence.

There is no circularity: the timestamp receipt is a sidecar over the already frozen release or manifest. A later revision can bind the receipt hash if a new signed object must refer to it. A timestamp does not prove creation at that moment, authorship, originality, or global first discovery. Multiple independently governed TSAs or transparency services can reduce single-service reliance, but policy is needed to say how their evidence is combined.

6 Implementation and Evaluation

The v1 prototype uses Python for deterministic fixture generation and a zero-dependency Node.js verifier for strict CBOR/COSE and profile validation; the two paths are intentionally separate. The package is designed for offline replay: locked dependencies are included, and the verification driver regenerates fixtures, verifies a standalone release, independently verifies the series profile, builds a manifest, and verifies that manifest.

A command-line tool (`acsd.py`) now turns the profile into an executable workflow: `keygen` (Ed25519), `init` (a release directory from a manuscript and a team file), `approve` (per-author COSE Sign1 endorsement), `finalize` (signature-complete package assembly with optional RFC 3161 timestamping), and `verify` (offline re-verification with explicit outcomes and non-claims). The canonical JSON layer is exercised by a 64-vector Python–Node differential harness that enforces accept/reject agreement across implementations; two divergences it surfaced (unpaired surrogate code points and tuples) are now rejected by both implementations, and three remaining object-layer float divergences are documented in the canonical specification.

The negative corpus includes: a signed series package with an insufficient structural rank; a signed package claiming an unsupported semantic edge; a release whose signed reference list disagrees with literal content tokens; a release with a shifted citation byte offset; and a correctly signed but non-canonical JSON release. All are rejected by the intended condition. A path resolver additionally rejects absolute paths, empty paths, and parent-directory escapes from profile references.

The current numbers are correctness coverage for a small demonstration corpus, not a scalability claim. The v2 evaluation adds parameterized release and series generators, records verification/manifest/rank time and peak memory, and runs a seeded Python–Node differential fuzzer. Until those experiments are complete, performance is unknown after this check.

The new Lean development models the PEC as concrete data structures rather than uninterpreted propositions: required and approved key lists, a claim policy with explicit non-claims, and an event list with consecutive sequences and predecessor-digest links. Its ten axioms-free theorems establish that a valid event chain has consecutive sequences (proved by induction), that approval is transitive

Table 1: Executable evaluation corpus. The first seven rows are inherited v1 fixture results; the last two are new v2 results reproducible from this release.

Object or check	Count / result
Standalone release versions	8 / 8 valid without series packages
Author endorsements	18 / 18 bound to the intended release
Signed series objects	7
Closed normal, attack, and indeterminate scenarios	13 / 13 expected outcomes
Independent profile checks	30 / 30 passed
Canonical JSON regression cases	11 / 11 Python–Node agreement
Public-manifest entries	113 / 113 verified
v2 differential canonical-JSON vectors	64 / 64 accept/reject agreement
New PEC theorems in the abstract core	10, no axiom dependency

and monotone, that a forbidden social claim is never a permitted outcome, and that acceptance implies full approval, a valid chain, and a non-amplifying policy. The earlier v1 core contains the broader citation-witness and rank arguments; those inherited results are not counted as new v2 PEC theorems here. These are proofs about an abstract verifier. They do not prove that the Python or Node parser refines the model, nor do they prove cryptographic security of the underlying algorithms.

7 Related Work

The profile builds on established foundations. PROV-DM supplies a general vocabulary for entities, activities, agents, derivations, and bundles [1]. Software Heritage identifiers provide intrinsic identifiers for software objects [2]; Trusty URIs provide hash-bearing immutable artifact identifiers [3]. COSE defines the signed binary envelope used for endorsements [4]. SCITT specifies a broader architecture for signed statements and transparency receipts [5]. RFC 3161 specifies the timestamp protocol [6].

ACSD does not claim to replace these systems or invent their primitives. Its narrow contribution is an executable scholarly profile that combines: self-contained anonymous paper releases; a separate series layer for mutual citations; exact text-token witnesses; explicit conflict predicates; and counterexamples against common overclaims about signatures, local clocks, and copied content.

8 Limitations and Future Work

Several extensions are intentionally deferred. First, relations across independently administered series require policy and an interoperable cross-series package model. Second, anti-equivocation evidence from transparency logs or gossip requires deployed service assumptions. Third, the parser, canonicalization, hash, and signature implementations have not been fully refined into the Lean model. Fourth, anonymous content publication does not hide repository-account, network, timing, or public-key linkage. Finally, richer citation parsing and selective identity linkage require separate privacy and semantic-design work.

These are reasons to state the present result narrowly, not reasons to weaken the useful evidence it already preserves.

Table 2: Capability boundary (not a performance comparison).

System	Primary capability	ACSD boundary
OpenTimestamps	Blockchain-anchored timestamp proofs	Optional time sidecar; no author-role or citation semantics
Software Heritage / SWHID	Persistent intrinsic identifiers for archived software objects	ACSD binds scholarly release roles, declarations and paper relations
Zenodo	Versioned archival record and DOI	ACSD’s byte-level policy and pseudonymous team assent remain explicit
RFC 3161	TSA-signed message-imprint time evidence	Exact not-after evidence only; never authorship or originality
ACSD	Release, governance, revision, witness and claim-policy profile	Composes these boundaries while retaining backend trust assumptions

9 Conclusion

ACSD turns a vague “anonymous signed paper” idea into a limited but executable evidence profile. It binds exact released objects to pseudonymous-key approvals, supports ordered roles and revisions, permits mutual scholarly citation without cyclic hash dependencies, and rejects several malformed or misleading artifacts. It leaves human identity, originality, and independent time as claims requiring additional evidence. This division of claims is the central design principle: preserve what cryptography can verify, and make the remaining social questions explicit.

References

- [1] W3C. *PROV-DM: The PROV Data Model*. W3C Recommendation, 2013. <https://www.w3.org/TR/prov-dm/>
- [2] Software Heritage. *SWHID Specification: Core Identifiers*, version 1.2. https://www.swhid.org/specification/v1.2/5.Core_identifiers/
- [3] T. Kuhn and M. Dumontier. *Trusty URIs: Verifiable, Immutable, and Permanent Digital Artifacts for Linked Data*. ESWC, 2015. <https://arxiv.org/abs/1401.5775>
- [4] J. Schaad. *CBOR Object Signing and Encryption (COSE): Structures and Process*. RFC 9052, 2022. <https://www.rfc-editor.org/rfc/rfc9052.html>
- [5] H. Birkholz et al. *An Architecture for Trustworthy and Transparent Digital Supply Chains*. RFC 9943, 2026. <https://www.rfc-editor.org/rfc/rfc9943.html>
- [6] C. Adams et al. *Internet X.509 Public Key Infrastructure Time-Stamp Protocol (TSP)*. RFC 3161, 2001. <https://www.rfc-editor.org/rfc/rfc3161.html>
- [7] OpenTimestamps. *A timestamp proof standard for blockchain anchoring*. Project documentation, accessed 2026-09-11. <https://opentimestamps.org/>